

Linux DNS Hardening & Verification Guide

Configuring systemd-resolved and NetworkManager for Total DNS Isolation

This guide outlines the technical steps required to audit, isolate, and lock down DNS routing on modern Linux systems utilizing `systemd-resolved` and `NetworkManager`. The goal is to enforce exclusive upstream routing to a secure provider (e.g., Quad9) and entirely eliminate DNS leaks through local Internet Service Provider (ISP) infrastructure.

Step 1: Auditing Global DNS Status

To verify the current operational status of the system name resolver, execute the standard resolution control utility:

```
resolvectl status
```

Critical Target Indicators:

- **resolv.conf mode:** Must report `foreign`. This ensures that an external configuration manager or static declaration is dictating global DNS topology, rather than raw local dhcp fallbacks.
- **Global Scope:** Secure upstream servers should be explicitly bound here. For a fully hardened environment, utilize Quad9:

```
Current DNS Server: 9.9.9.9
DNS Servers: 9.9.9.9 149.112.112.112 2620:fe::fe 2620:fe::9
```

Step 2: Identifying and Resolving Interface Leaks

Even with clean Global parameters, active hardware interfaces (e.g., `eno1` or `wlan0`) can independently acquire DNS allocations directly from local routers via DHCP or IPv6 Router Advertisements (RA). This creates a split-horizon configuration where lookup requests can leak to the ISP.

If your interface link block exposes legacy infrastructure or provider IPs (such as standard cable/fiber carrier strings), the hardware configuration must be modified to ignore automatic downstream DNS metrics.

Step 3: NetworkManager Enforcement via CLI

When system graphical environments conflict with complex dual-stack manual changes, `nmcli` provides definitive control. Run the following parameters to mandate address-only DHCP leases, locking out automatically assigned DNS vectors:

1. Identify Connection Tokens

```
nmcli connection show
```

Locate the specific target token matching your physical adapter device under the **DEVICE** string mapping (e.g., **"Wired connection 1"**).

2. Strip DHCP DNS and Force Overrides

```
nmcli connection modify "Wired connection 1" ipv4.ignore-auto-dns yes ipv6.ignore-  
auto-dns yes ipv4.dns "9.9.9.9,149.112.112.112" ipv6.dns "2620:fe::fe,2620:fe::9"
```

3. Reinitialize the Network Interface

```
nmcli connection up "Wired connection 1"
```

Step 4: Final Verification

Re-run **resolvectl status** to evaluate the network hierarchy. A successfully hardened configuration must completely match the structural profile below:

```
Global
    Protocols: +LLMNR -mDNS -DNSoverTLS DNSSEC=no/unsupported
    resolv.conf mode: foreign
    Current DNS Server: 9.9.9.9
    DNS Servers: 9.9.9.9 149.112.112.112 2620:fe::fe 2620:fe::9

Link 2 (eno1)
    Current Scopes: DNS LLMNR/IPv4 LLMNR/IPv6
    Protocols: +DefaultRoute +LLMNR -mDNS -DNSoverTLS DNSSEC=no/unsupported
    Current DNS Server: 9.9.9.9
    DNS Servers: 9.9.9.9 149.112.112.112 2620:fe::fe 2620:fe::9
    Default Route: yes
```

Verification Success Metrics

Your physical link layer is now strictly mirror-mapped to your secure global DNS provider. Every transactional network lookup path is constrained, leaving zero valid vectors for ISP leakage, passive collection, or metadata tracking.